

Results

This section reports repository facts produced by the meta-project's live introspection pass. Test counts, module inventories, pipeline-stage counts, and figure data are injected from generated artifacts. Runtime and achieved-coverage claims are intentionally omitted when no versioned benchmark or coverage artifact backs them.

Multi-Project Pipeline Execution I

The `./run.sh` interactive orchestrator can execute the public roster through the shared DAG. The table below is a structural snapshot, not a timing benchmark: counts come from repository introspection, and the coverage column states each exemplar's declared policy floor rather than an unversioned observed percentage.

Multi-Project Pipeline Execution I

Project	Effective core stages ¹	Discovered tests	Declared project floor
template_code 8		246	90%
_project			
template_pros 8		151	90%
e_project			
template_auto 8		307	90%
research_proj			
ect			

¹“Core-only” excludes LLM-tagged and other opt-in stages according to the YAML stage tags. A fresh run must be used to establish completion status or wall-clock performance for a particular machine and dependency set.

Infrastructure Test Suite I

Infrastructure Test Suite I

Metric	Value
Test files	589+
Total tests	~10,163
Infrastructure coverage gate	≥60% configured floor
Prohibited mock-framework imports	Checked by the static no-mocks gate

Infrastructure Test Suite I

Exercises Pandoc/XeLaTeX paths, steganography hashing, telemetry, YAML-driven pipeline DAG resolution, HTTPS-bound optional suites (`pytest-httpserver`), and local Ollama-gated subsets.

Infrastructure Module Inventory I

The introspection module (`template_template.core.introspection`) emits the authoritative table below—every row reflects `discover_infrastructure_modules(REPO_ROOT)`.

Infrastructure Module Inventory II				
Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
autoresearch	11			build_autoresearch_plan, readiness validation CLI
benchmark	4			Template harness scoring
config	0			+ comparative gates
core	133			Repository defaults + hardened templates
				get_logger,
				load_config,
docker	0			TemplateError
				Containerisation
doctor	14			scaffolding
				Checkout
documentation	23			diagnose/fix/undo repairs
				FigureManager, generate
fonds	6			_glossary
llm	55			—
				Ollama helpers,
				sanitization, review +
				translation pipelines
logrotate.d	0			Rotation snippets
				(documentation-first)
methods	8			build_methods_orchestration_plan,
				methods-stage contracts +
				validation
orchestration	12			PipelineRunner, entry
				point for ./run.sh
project	42			discover_projects,
				workspace management
prose	9			Markdown readability +
				prose tooling
provenance	8			—
publishing	84			Zenodo. executable

All 28 enumerated subdirectories carry Tier-1/README.md and Tier-2/AGENTS.md coverage wherever the Documentation Duality standard applies; subsets ship Tier-3 SKILL.md descriptors for MCP routing (infrastructure/skills manifest generation).

Agentic Skill Documentation Coverage I

Agentic Skill Documentation Coverage I

Layer	Role
System prompts	Root CLAUDE.md, README policy
Structural	AGENTS.md directories
Skills	Optional SKILL.md manifests + generated .cursor/skill_manifest.json
PAI capsule	Repository level PAI.md narratives

Agentic Skill Documentation Coverage I

411+ Markdown shards under docs/ capture operational knowledge without duplicating auto-generated inventories.

DAG Reference (Declarative Executor) I

Stages below `mirror pipeline.yaml` (executor-topological order—not strict numeric script filenames). Scripts live under `scripts/`.

DAG Reference (Declarative Executor) I

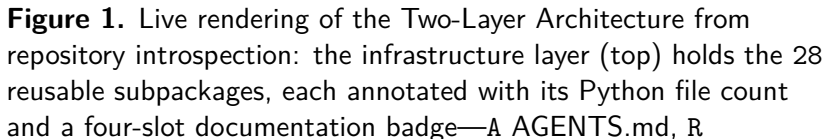
Name	Typical script / method	Responsibility	Failure semantics
Clean Output	<code>_run_clean_outputs</code>	Deletes stale output/ trees	Blocking
Directories			
Environment Setup	<code>scripts/pipeline/stage_00_setup.py</code>	Validates tooling, PYTHONPATH scaffolding	Blocking
Infrastructures	<code>scripts/pipeline/stage_01_test.py</code>	Infra pytest + coverage gates	Tunable thresholds
Tests	<code>--infra-only</code>		
Project	<code>scripts/pipeline/stage_01_test.py</code>	Project pytest + coverage gates	Zero failures default
Tests	<code>--project-only</code>		
Project Analysis	<code>scripts/pipeline/stage_02_analysis.py</code>	Executes projects/<name>/scripts/*.py	Blocking
PDF	<code>scripts/pipeline/stage_03_render.py</code>	Pandoc → XeLaTeX manuscripts	Blocking
Rendering			
Output	<code>scripts/pipeline/stage_04_validate.py</code>	Structural PDF/markdown probes	Blocking / warnings
Validation			
LLM	<code>scripts/pipeline/stage_06_llm_review.py</code>	Local Ollama reviews	Skippable / exit2 tolerated
Scientific Review	<code>--reviews-only</code>		
LLM	<code>scripts/pipeline/stage_06_llm_review.py</code>	Optional translations	Skippable
Translations	<code>--translations-only</code>		
Copy Outputs	<code>scripts/pipeline/stage_05_copy.py</code>	Mirrors deliverables → output/<project>/	Soft-fail surfaced in logs

scripts/pipeline/stage_07_executive_report.py is **multi-project orchestration glue** invoked after iterating active projects—not a tenth DAG node for single-repo runs (execute_pipeline.py).

Steganographic Performance Boundary I

The steganography path exposes metadata injection, SHA-256 hashing, overlay generation, and optional QR-code insertion as separately testable operations. This revision makes no latency claim because the project does not currently generate a versioned benchmark artifact recording hardware, input PDF, warm-up policy, repetitions, and dispersion. A future performance table should be compiled from such an artifact rather than copied from an ad-hoc workstation run.

Rendered via `projects/templates/template_template (generate_manuscript_metrics.py → injected tokens such as 28)`.
Architecture figures stem from `template_template.figures.architecture_viz`—font sizes constrained by §QA.



Self-Referential Analysis I

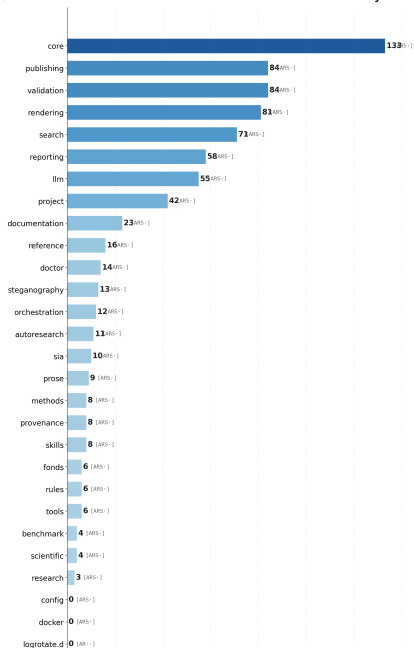
[ARSP]. A YAML DAG arrow connects it to the project layer (bottom) of public exemplars labelled with chapter and test counts. The takeaway: documentation-duality coverage is near-uniform across the infrastructure, and every box was placed from the same live data the prose cites.

Pipeline DAG (17 declared stages)



Figure 2. Pipeline DAG with 17 YAML-declared stages (core, LLM, ebook, docxplus, metadata, bundle, archival tags).

Self-Referential Analysis I



Comparative Feature Analysis I

Figure 4 summarizes the Appendix F capability matrix.

Comparative Feature Analysis II



Figure 4. 14×10 heatmap annotated in appendix text—green full native capability, yellow partial / plugin-hosted, red — unavailable. Rows group *Core Pipeline*, *Quality & Security*, then *Ecosystem*.

Comparative Feature Analysis I

- ¹ Nextflow lineage records exist at workflow scope, not per rendered PDF citation graph.
- ² DVC: content-addressed artifacts without native prose rendering.
- ³ DVC: remote object stores (S3, GCS, Azure) without turnkey CI manuscript gates.

Test Quality Metrics I

Test Quality Metrics I

- ▶ **Zero mocks:** repository policy bans `unittest.mock` / patching frameworks in tests.
- ▶ **Filesystem + subprocess realism:** ephemeral directories + actual CLI binaries.
- ▶ **HTTP realism:** infra suites favour `pytest-httpserver`; literature tests hit recorded fixtures.
- ▶ `template_code_project` focuses on numerical reproducibility assertions.
- ▶ `template_autoresearch_project` exercises the AutoResearch readiness planner (`infrastructure/autoresearch/`). `template_search_project` exercises literature-search workflows.